Learning Kinematic Reference Trajectories for a Series-Elastic Transfemoral Prosthesis in a Muscle-Driven Simulation

Tommaso Scagliarini Affiliation to be added
Email to be added

Abstract*—This manuscript develops and validates an architecture for reinforcement-learned prosthetic reference generation in a muscle-driven OpenSim simulation. The learned policy is designed to generate knee and ankle kinematic references for a series-elastic transfemoral prosthesis, while a band-limited reference interface, a fixed cascade/PI controller, and a CMC-like biological tracking loop determine whether those references are physically feasible. Current experiments use imitation pretraining as the first stage toward ex-novo prosthetic trajectory generation. *Replace this placeholder abstract after the final baseline ladder, multi-seed results, and quantitative physical-validity metrics are available.

***Index Terms*—Reinforcement learning, prosthetics, trajectory generation, series-elastic actuators, musculoskeletal simulation, OpenSim.**

I. Introduction

TODO: draft Section "Introduction" in a separate file under `sections/`.

II. Related Work

TODO: draft Section "Related Work" in a separate file under `sections/`.

III. Simulation Testbed

TO BE REVIEWED

This section describes the physical testbed used both to evaluate learned prosthetic reference trajectories and to validate the prosthetic control chain. The learned policy does not directly command muscles or motor torques. Instead, it provides a prosthetic kinematic reference that is tracked through the controllers. On the biological side, experimental kinematics are used as prescribed tracking references: the simulator (based on OpenSim's *Computed Muscles Control*) computes the generalized force demand and distributes it through muscle recruitment and residual actuators. Therefore the prosthetic and biological coordinates are treated by two distinct control paths. The prosthetic side is driven by the learned reference and SEA controller, whereas the biological side is tracked from experimental kinematics through muscle-driven recruitment. Both sides remain coupled in the same forward-dynamics OpenSim simulation. This separation is central to the paper: a learned trajectory is meaningful only if it can be realized by a physical series-elastic prosthesis inside a muscle-driven gait simulation.

A. Series-Elastic Prosthesis Model

Series-elastic actuators consist of an electric motor connected to the joint through a torsional spring in series. Thus, the joint is not directly driven by the motor, but is actuated through the deflection of the spring. SEAs were selected to actuate the transfemoral prosthesis for two main reasons:

1) The spring can store and release energy during the gait cycle. This may improve energy efficiency compared with conventional motor actuation and provides intrinsic compliance, allowing the actuator to attenuate impacts during gait.
2) SEAs are mechanically simpler than variable stiffness actuators.

Each SEA was implemented as a custom OpenSim plugin. This choice preserves compatibility with the OpenSim software suite.

The SEA plant is described by two physical motor-side state variables:

$$\theta_m, \quad \omega_m, \tag{1}$$

where θ_m is the motor-side angle, ω_m is the motor angular velocity, joint coordinate angle is denoted by θ_j. The elastic torque transmitted through the series spring is

$$\tau_s = K(\theta_m - \theta_j), \tag{2}$$

where K is the spring stiffness.
The SEA dynamics are

$$\dot{\theta}_m = \omega_m, \qquad J_m\dot{\omega}_m = \tau_m - \tau_s - B_m\omega_m. \tag{3}$$

Here J_m is the motor inertia, B_m is the motor damping, and τ_m is the motor-side torque.
Thus, the prosthetic joint is actuated by the torque transmitted through the elastic element, rather than being directly driven by the motor. The resulting torque-generation dynamics are therefore determined by the coupled behavior of the motor and compliant transmission, with motor inertia, damping, and spring stiffness governing the rate at which torque can be developed and delivered to the joint.
The actuators are dimensioned as follows:

Thus, the prosthetic joint is actuated by the torque transmitted through the elastic element, rather than being directly driven by the motor. The resulting torque-generation dynamics are therefore determined by the coupled behavior of the motor and compliant transmission, with motor inertia, damping, and

SEA	K N m/rad	J_m kg m^2	B_m N m s/rad	F_{opt} N m
Knee	321	0.01	0.1	100
Ankle	500	0.01	0.1	250

TABLE I
SEA PARAMETERS ADOPTED.

spring stiffness governing the rate at which torque can be developed and delivered to the joint.

Figure placeholder. Fig. 2 should show the two-body SEA schematic with θ_j, θ_m, K, B_m, J_m, τ_s, and τ_m.

B. CMC-Like Muscle-Driven Simulation

The prosthetic model has been evaluated using a custom simulator based on OpenSim's *Computed Muscle Control* (CMC) tool. OpenSim Computed Muscle Control is a forward-dynamics tracking method that computes the muscle excitations required for a musculoskeletal model to follow a prescribed kinematic trajectory. At each time step, the simulated state is compared with the reference motion, desired accelerations are generated through a tracking controller, and muscle recruitment is solved to produce the generalized forces needed to realize those accelerations. Reserve actuators may be included as residual support when the modeled muscles cannot fully satisfy the required dynamics. The limitation of the original CMC is that it does not allow to impose a custom control law on a subset of model coordinates.

Figure placeholder. Fig. 3 should show the CMC loop

The integration step is $1\,\text{ms}$, and the validated forward-dynamics path uses the C++ SEA plugin with the `rk4_bypass` integration scheme.

The kinematic reference is read from the inverse-kinematics file and preprocessed before it is used by the controller. The trajectory is resampled on a $1\,\text{ms}$ grid and low-pass filtered with a zero-phase Butterworth filter at $6\,\text{Hz}$. This preprocessing keeps the spline derivatives usable for acceleration-level tracking and avoids injecting high-frequency IK noise into the inverse-dynamics and static-optimization stages.

At each control instant, the CMC-like biological tracking loop computes desired accelerations for the non-prosthetic coordinates:

$$\ddot{q}_{\text{des}} = \ddot{q}_{\text{ref}} + K_{p,b}(q_{\text{ref}} - q) + K_{d,b}(\dot{q}_{\text{ref}} - \dot{q}). \tag{4}$$

The default biological gains are $K_{p,b} = 100$ and $K_{d,b} = 20$, with lower translation gains on the pelvis coordinates $(25/10)$ to reduce root-coordinate drift without forcing the muscle-driven joints through the reserve actuators. The prosthetic coordinates are excluded from this biological outer loop and from the biological static-optimization problem; they are handled by the SEA controller.

The generalized force request is computed using a zero-actuator inverse-dynamics projection. First, all muscle, reserve, and SEA controls are removed. The baseline acceleration $\ddot{q}_0$ is computed at the current state with the actuator contributions zeroed. The acceleration deficit is then projected through the configuration-dependent mass matrix:

$$\tau = M(q)\left(\ddot{q}_{\text{des}} - \ddot{q}_0\right). \tag{5}$$

The resulting generalized-force vector is split into a biological component and a prosthetic component:

$$\tau_{\text{bio}} = \tau[\mathcal{I}_{\text{bio}}], \qquad \tau_{\text{pros}} = \tau[\mathcal{I}_{\text{pros}}]. \tag{6}$$

Only τ_{bio} is passed to the muscle recruitment step. The prosthetic component is retained as a diagnostic inverse-dynamics oracle, but it is not used as a feed-forward command in the current prosthetic controller.

Biological recruitment is solved with a muscle-first static-optimization problem. The optimizer distributes τ_{bio} across muscle activations and reserve actuator controls:

$$\begin{aligned} \min_{a,u_r} \quad & \sum_i a_i^2 + w_r \sum_j u_{r,j}^2 \\ \text{s.t.} \quad & A_m(q,\dot{q},l_f)(a - a_{\min}) + R_r(u_r \odot F_{\text{opt},r}) = \tau_{\text{bio}}, \\ & a_{\min} \le a_i \le a_{\max}, \\ & -u_{r,\max} \le u_{r,j} \le u_{r,\max}. \end{aligned} \tag{7}$$

The reserve weight is set to $w_r = 10^6$. Thus, reserves remain available to close unmodeled or infeasible residual directions, but they are strongly discouraged from becoming the primary tracking mechanism. In the analysis, reserve usage is treated as a fidelity gauge rather than as a success metric: low reserve usage indicates that the simulated motion is supported by the model's muscles and prosthesis, whereas high reserve usage exposes a dynamic inconsistency, a contact-model deficit, or an unmodeled actuation demand.

The CMC-like loop therefore provides two pieces of information that are essential for evaluating learned prosthetic trajectories. First, it determines whether the biological side can remain muscle-driven while the prosthesis follows the generated reference. Second, it makes reserve and residual loads available as quantitative diagnostics of physical validity. This prevents the RL policy from being judged only by kinematic tracking or reward return.

Figure placeholder. Fig. 1 should include the full testbed flow: IK spline → biological outer-loop accelerations → zero-actuator inverse dynamics → static optimization → prosthetic cascade → SEA plugin → OpenSim state update.

Table placeholder. Table IV should summarize the AB06 model, simulation window, time step, prosthetic coordinates, biological tracking gains, static-optimization weights, and GRF mode.

C. Cascade/PI Prosthetic Reference Tracking

The two SEA coordinates are driven by a fixed high-level prosthetic tracking controller implemented in Python. This controller receives the prosthetic kinematic reference $(q_{\text{ref}}, \dot{q}_{\text{ref}})$, compares it with the current prosthetic coordinate state $(q, \dot{q})$, and computes the normalized SEA command u sent to the C++ plugin. The current validated mode is a position-to-velocity cascade followed by an inner velocity PI loop.

For each prosthetic coordinate, the outer proportional position loop computes a desired cascade velocity:

$$\dot{q}_{\text{cas}} = \dot{q}_{\text{ref}} + K_{p,o}(q_{\text{ref}} - q). \tag{8}$$

The inner loop then converts the velocity tracking error into a torque command:

$$e_v = \dot{q}_{\text{cas}} - \dot{q}, \tag{9}$$

$$\tau_{\text{cmd}} = K_{p,i} e_v + K_{i,i} \int e_v \, dt. \tag{10}$$

The integral contribution is clamped per joint to prevent wind-up. Finally, the controller normalizes the requested torque by the corresponding SEA optimal force:

$$u = \text{clip}\left(\frac{\tau_{\text{cmd}}}{F_{\text{opt}}}, -1, 1\right). \tag{11}$$

This u is the only prosthetic command injected into OpenSim. The C++ plugin converts it into a desired spring torque,

$$\tau_{\text{ref}} = F_{\text{opt}} u. \tag{12}$$

Inside the plugin, the actuator drive closes a motor-side spring-torque servo. Let ξ denote the integrated spring-torque tracking error. In the non-impedance mode used by the validated model, the raw motor torque is

$$\tau_{m,\text{raw}} = \tau_{\text{ref}} + K_p(\tau_{\text{ref}} - \tau_s) + \text{sat}(K_i \xi, \pm L_i) - K_d \omega_m, \tag{13}$$

with

$$\dot{\xi} = \tau_{\text{ref}} - \tau_s. \tag{14}$$

The integral contribution is limited to $\pm L_i$, and the integrator is frozen when either the integral contribution or the motor torque is saturated in the direction of the error. The motor torque applied to the rotor is

$$\tau_m = \text{clip}(\tau_{m,\text{raw}}, -500, 500)\,\text{N m}. \tag{15}$$

This τ_m drives the SEA dynamics in Eq. (3); the joint still receives the spring torque τ_s defined in Eq. (2).

The cascade gains used in the current configuration are reported in Table III. The knee uses $K_{p,o} = 18.85\,\text{s}^{-1}$, $K_{p,i} = 29.2\,\text{N m s/rad}$, $K_{i,i} = 1377\,\text{N m/rad}$, and a $50\,\text{N m}$ inner integral torque limit. The ankle uses $K_{p,o} = 47.125\,\text{s}^{-1}$, $K_{p,i} = 2.8275\,\text{N m s/rad}$, $K_{i,i} = 213\,\text{N m/rad}$, and a $200\,\text{N m}$ inner integral torque limit. The embedded actuator-drive gains are $(K_p, K_d, K_i) = (18, 11, 190)$ for the knee and $(11.3, 11, 123)$ for the ankle, with $L_i = 100\,\text{N m}$ for both joints.

This controller is deliberately not a learned low-level policy. It is a fixed, validated tracking interface between the generated kinematic reference and the SEA plant. Its role is to expose the correct failure mode to the trajectory generator: when the reference is too aggressive, the controller cannot hide that fact. It produces larger u, larger motor torque demand, increased spring tracking error, or saturation. Conversely, a feasible reference remains trackable without changing the low-level SEA parameters. This is why the reference governor introduced in Section IV is part of the action interface rather than a post-hoc smoothing operation.

Table placeholder. Table III should report cascade gains, cascade integral limits, actuator-drive gains, actuator-drive integral and motor-torque limits, reference-governor velocity/acceleration/jerk limits, and the normalized command limits.

Before moving this section to a final submission template, verify all reported numerical gains against the final model/config snapshot used for the experiments, and decide whether "series-elastic actuator" or "SEA" is used as the dominant term.

IV. RL Trajectory Generator

TODO: draft Section "RL Trajectory Generator" in a separate file under `sections/`.

V. Experimental Setup

TODO: draft Section "Experimental Setup" in a separate file under `sections/`.

VI. Results

TODO: draft Section "Results" in a separate file under `sections/`.

VII. Discussion

TODO: draft Section "Discussion" in a separate file under `sections/`.

VIII. Limitations and Future Work

TODO: draft Section "Limitations and Future Work" in a separate file under `sections/`.

IX. Conclusion

TODO: draft Section "Conclusion" in a separate file under `sections/`.